

DE 424: Wumpus Trajectory Tracking Model

Amy McDermott (26911264)

May 14, 2026

Declare that this work presented is your own work. Mention any sources that contributed to work or report
I declare that this report is my own work. This report was written using LaTeX and thus Gemini was used to assist in certain formatting. This includes help to format code listings, mathematical equations and image display. The mathematical derivations and interpretation of results are my own work.

ECSA graduate attributes

Discuss how the GAs were assessed in the module + which parts of the report this is documented. State HOW and WHERE they were satisfied This section may (and should) be brief, but should discuss all aspects of the GAs being assessed. For example, GA1 partially assesses your ability to research literature as part of problem solving, which means that your report must mention and cite (at least) a few sources - can include prescribed resources. Please consult the module framework for a more detailed description of these two GAs as well as how they are assessed in this module.

GA 1: Problem solving

GA 2: Application of scientific and engineering knowledge

1 Problem Description

This project addresses an object tracking task within a discrete grid world inhabited by a latent entity known as a wumpus. The objective is to determine the trajectory of the wumpus across multiple time steps based on inaccurate measurements. At each time step the wumpus attempts to move to an adjacent cell (up, down, left, or right) with equal probability. If the target cell is within the grid boundaries, the wumpus transitions to that location; otherwise it remains in its current position. The exact location of the wumpus is never directly observed. Instead, every cell in the grid generates a binary detection value at each time step, parameterised by:

- True Positive (p_w): The probability that the cell containing the wumpus correctly generates a detection.
- False Positive (p_c): The probability that a cell not containing the wumpus generates an incorrect detection (clutter measurement).

Given the stochastic grid detection data across multiple time steps, the task is to design, implement and evaluate a Probabilistic Graphical Model (PGM) solution. This involves constructing a model that accurately captures the wumpus's transition and observation logic to infer its latent trajectory across various datasets.

2 Solution Design

The solution was developed iteratively to accommodate the increasing complexity and varying characteristics of each dataset. The PGM design process followed a structured methodology: defining the random variables, defining the model, and constructing the model factors. This process is detailed below for each attempted dataset. (note: this section must show model structure and show all factors. State and motivate all design choices)

2.1 Dataset 1

The initial model was designed for a 5×5 grid over a series of $T = 10$ time steps. In this case, the parameters $p_w = 0.95$ and $p_c = 0.05$ are known constants.

2.1.1 Random Variable Definition

A critical design choice was made to represent the wumpus’s location using a single latent RV, W^t , instead of separate X^t and Y^t coordinates. The grid is mapped to a 1-dimensional vector where each cell index $i \in \{0, \dots, 24\}$ corresponds to a unique (x, y) coordinate. This approach offers several advantages. Primarily, it enhances inference efficiency by reducing the total number of RVs within the cluster graph, which leads to more efficient message passing and overall faster convergence. Furthermore, it reduces complexity by simplifying factor construction and avoiding the complexities of coordinating the dependencies between spatial coordinates. The wumpus’s movement constraints are more easily enforced within a transition matrix; for example, the probability of illegal moves, such as diagonal steps, can be explicitly set to zero within one factor. The RVs are defined as follows:

- $W^t \in \{0, 1, \dots, 24\}$: The latent position of the wumpus at time t .
- $D_i^t \in \{0, 1\}$: The binary detection status of cell i at time t , where 1 indicates a detection.

2.1.2 Model Definition

The wumpus tracking task is formulated as a Hidden Markov Model (HMM), a framework used to infer a sequence of hidden states from observable outputs [1]. The latent trajectory of the wumpus ($W^0, W^1, \dots, W^T$) is modeled as a first-order Markov process, $P(W^t | W^{t-1})$, where every future state depends on the current state, not on the sequence of states preceding it. The observable detection variables are emissions that are generated from each state $p(D_i^t | W^t)$. A HMM is a subclass of Bayesian Networks, used for sequential data. The model is illustrated below in Figure 1.

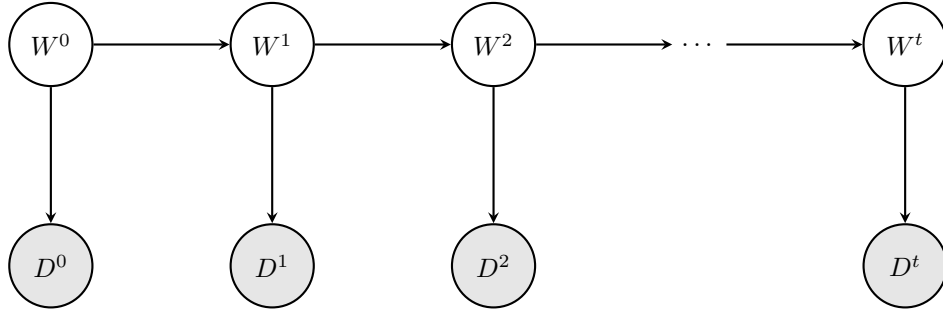


Figure 1: Bayesian Network structure of the HMM tracking solution.

2.1.3 Factor Definition

The model is governed by two primary factors. The first factor models the transition model of the wumpus over time as a first-order Markov process $P(W^t | W^{t-1})$. This factor captures the entity’s movement across adjacent cells, where the wumpus attempts to move up, down, left, or right with an equal probability of 0.25 per direction. To enforce the physical boundaries of the grid, the wumpus remains in its current position if it attempts to move outside the grid world. Consequently, the probability of the wumpus remaining stationary ($P(W^t = i | W^{t-1} = i)$) is determined by its proximity to the boundaries:

- Interior cells: The wumpus always moves to an adjacent cell with a probability of 0.25. Thus, the probability of remaining in place is 0.
- Edge cells: With the wumpus having one direction blocked, the probability of remaining in place is 0.25.

- Corner cells: With the wumpus having two blocked directions, the probability of remaining in place increases to 0.5.

For a 5×5 grid, the stochastic wumpus trajectory results in a sparse 25×25 transition matrix that ensures that the inferred wumpus movement adheres to the constraints of the environment. This transition factor is invariant across all time steps $t \in \{1, \dots, T\}$ since the wumpus's rules of movement do not change as time progresses.

Table 1: Illustrative sample of the Transition Probability Factor $P(W^t | W^{t-1})$ for a 5×5 grid.

W^{t-1}	W^t	$P(W^t W^{t-1})$	Description
0	0	0.50	Stay (Blocked Up and Left)
0	1	0.25	Valid Move (Right)
0	5	0.25	Valid Move (Down)
0	6	0.00	Illegal Move (Diagonal)
12	12	0.00	Stay (No Boundaries Hit)
12	13	0.25	Valid Move (Right)
...	...	...	...

Secondly, a factor that links the latent state W^t to the observed grid-wide detection data $\mathbf{D}^t$ is designed $P(\mathbf{D}^t | W^t)$. This factor is parameterised by the detection probability p_w and the clutter probability p_c , which accounts for sensor inaccuracies across all cells in the grid. In the context of HMMs, this factor defines the likelihood of a specific observation being generated from a particular hidden state, otherwise known as an emission probability.

Table 2: Illustrative sample of the Detection Factor for a single cell i at time t .

W^t	D_i^t	$P(D_i^t W^t)$	Description
i	1	p_w (0.95)	True Positive (Correct Detection)
i	0	$1 - p_w$ (0.05)	False Negative (Missed Detection)
$j \neq i$	1	p_c (0.05)	False Positive (Clutter)
$j \neq i$	0	$1 - p_c$ (0.95)	True Negative (Correct Non-Detection)

2.1.4 Cluster Graph and Inference

The topology of an HMM naturally forms an acyclic graph. (source). Due to this, the Bayesian Network shown in Figure 1 is converted to a Junction Tree cluster graph using the built-in functionality of the **emdw** library. This structure guarantees exact inference through a single forward-backward message-passing schedule, resulting in a balance between computational efficiency and inference accuracy. Due to the tree structure, the complexity scales linearly, $O(T)$, with the number of time steps. The factors and cluster graph are shown below:

- $P(W^t | W^{t-1})$ is assigned to a cluster with the scope $\{W^{t-1}, W^t\}$.
- $P(\mathbf{D}^t | W^t)$ is assigned to a cluster with scope $\{W^t\}$.

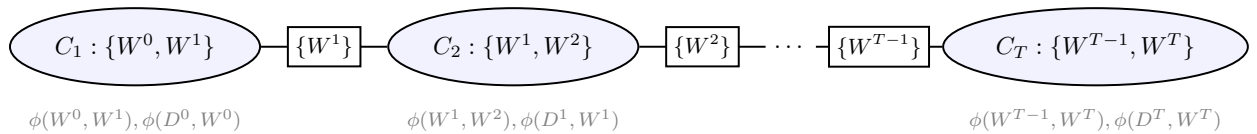


Figure 2: Cluster graph for exact inference.

For the trajectory inference task, Maximum A Posteriori (MAP) inference using the Max-Product algorithm was selected over Marginal inference. This choice is motivated by the fact that MAP inference

computes the most likely trajectory $(W^0, W^1, \dots, W^T)$ which results in a path that is globally consistent and adheres to the wumpus’s movement constraints as defined in the transition model. The MAP estimates represent a valid movement of the wumpus through the grid world. In contrast, Marginal inference using the belief-propagation algorithm identifies the most likely cell for the wumpus at each discrete time step independently. This may lead to a sequence of locations that are infeasible if certain cells have high local probabilities.

2.2 Dataset 2

Extends upon dataset1 - we can see if model can expand for larger grid sizes. Dataset 2 uses a 20×20 grid. This results in 400 possible states. The transition factor for Dataset 2 will therefore be a 400×400 matrix. $O(T)$ scales.

3 Code Implementation

3.1 Dataset 1

The model uses two primary vectors for RV management: `W_rvs` for the latent wumpus positions and `D_rvs` for observed detections. Storing the random variable IDs in these vectors allows efficient looping during the factor construction phase, as specific time dependent RVs (W^{T-1} and W^t) can be accessed directly via their vector indices. `D_rvs` is a 2D vector which indexes one RV for each time step and cell location.

```
1 vector<T> W_rvs(T_max);
2 vector<vector<T>> D_rvs(T_max, vector<T>(25));
```

Listing 1: Declaration of latent and observable random variable vectors.

The transition model is implemented by iterating through each time step and constructing a transition factor $P(W^t | W^{t-1})$ that connects every current state to its previous. The implementation logic follows a few key steps. Firstly, within each iteration of the temporal loop, the unique IDs for the current and previous wumpus positions are retrieved from the `W_rvs` vector. Because the grid is represented using a 1D index (0 to 24), the code converts these indices back into (x, y) coordinates to facilitate the check for valid adjacent moves.

```
1 int x = i % width; // x-coordinate
2 int y = i / width; // y-coordinate
```

Listing 2: Conversion of grid indices to (x, y) coordinates

Coordinate offsets (dx and dy) are used to determine all potential subsequent wumpus location. To handle the grid boundaries, the proposed wumpus position is checked if it stays within the grid dimensions. If a move is valid, it is assigned a probability of 0.25; and if a move would go out of bounds, the 0.25 probability is instead accumulated. This ensures that the probability of staying in a cell correctly reflects how many boundaries that the cell is touching (e.g., 0.5 in a corner).

```
1 // Define possible grid moves (up, down, left, right)
2 int dx[] = {0, 0, -1, 1};
3 int dy[] = {1, -1, 0, 0};
4 for (int move = 0; move < 4; move++) {
5     int new_x = x + dx[move]; // wumpus new x-coordinate
6     int new_y = y + dy[move]; // wumpus new y-coordinate
7     if (new_x >= 0 && new_x < width && new_y >= 0 && new_y < width) {
8         // Valid transition
9         int new_i = new_y * width + new_x; // Convert back to cell index
10        transitionProbs[[i, new_i]] = 0.25;
11    } else {
12        // Invalid transition
13        stay_prob += 0.25; // Accumulate probability of staying in cell
```

```

14 }
15 }
16 transitionProbs[{i, i}] = stay_prob; // Prob of wumpus staying in the same cell

```

Listing 3: Transition factor logic.

A `std::map` is used to store only the non-zero probabilities to prevent creating a massive 25×25 table with many zeros which consequently improves computational performance. These sparse entries are then used to instantiate an `emdw` Discrete Table (DT) called `ptrTransition`, which is then stored in a vector of factor pointers for later use in the cluster graph.

```

1 transitionFactors.push_back(ptrTransition);

```

Listing 4: Vector of transition factor pointers

```

1 // loc is the location of the specific cell we are building a factor for
2 for (int loc = 0; loc < num_cells; loc++) {
3     int D_curr = D_rvs[t][loc]; // Extract index of D RV at time t and grid location
4     map<vector<T>, FProb> detectionProbs;
5     // w_pos is the latent position of the wumpus
6     for (int w_pos = 0; w_pos < num_cells; w_pos++){
7         if (w_pos == loc) {
8             // Detection location is equal to the wumpus position
9             detectionProbs[{1, w_pos}] = pw; // Detection
10            detectionProbs[{0, w_pos}] = 1.0 - pw; // Missed detection
11        } else {
12            detectionProbs[{1, w_pos}] = pc; // Clutter
13            detectionProbs[{0, w_pos}] = 1.0 - pc; // No detection
14        }
15    } // end of wumpus location loop
16 }

```

Listing 5: Detection factor initialisation.

1. Create factor to store all detection and transition factors 2. Create a map to contain all observed RV values and load evidence from external files into `obsv` 3. Create cluster graph with `emdw` 4. perform message passing with `emdw` functionality

```

1 ClusterGraph cg(ClusterGraph::BETHE, factorPtrs, obsv);
2 map<Idx2, rcptr<Factor> > msgs;
3 MessageQueue msgQ;
4 unsigned nMsgs = loopyBP_CG(cg, msgs, msgQ);

```

Listing 6: Cluster graph and message passing

Perform MAP inference on Wumpus location for all time steps and save results to file `wumpus_location1.txt` `rcptr<Factor> beliefT = queryLBP_CG(cg, msgs, W_rvs[t]); beliefT->normalize();` // Query the graph For every cell for time step `t`, we perform: to extract highest max marginal (will = 1 in this case) double likelihood = `beliefT->potentialAt(W_rvs[t], (T)cell_idx);`

if (likelihood > max_likelihood) max_likelihood = likelihood; best_cell = cell_idx; Convert the bestCell index back to (row, col) and output to file `wumpus_location1.txt`

Notes to add - MAP inference: Specify operators to the factor, so that when `marginalize()` operator is called, maximisation occurs instead. Now, when you call the member function `marginalize`, it will perform maximisation instead, and when you call the member functions `normalize` and `inplaceNormalize`, it will perform normalisation appropriate for MAP inference (i.e., the factor will be scaled such that the maximum value is 1).

Creating the factors this way, you can now perform MAP inference using exactly the same code as for marginal inference.

The result of MAP message passing is a set of so-called max-marginal beliefs – one for each cluster (this is similar to the cluster beliefs of BP). A max-marginal belief contains a likelihood value for each combination

of RVs in the scope of its cluster. If there is a single most likely global combination of RVs, extracting this combination is easy: the most likely (local) combination of each max-marginal belief is consistent with the most likely global combination

for dataset 2: updated how to create locationDom updated all parameters (num_cells) etc changed reading in data function to account for the different naming convention

4 Analysis of Results

Description of how you processed the output of solution, the presentation of results and analysis thereof. Report both qualitative and quantitative results.

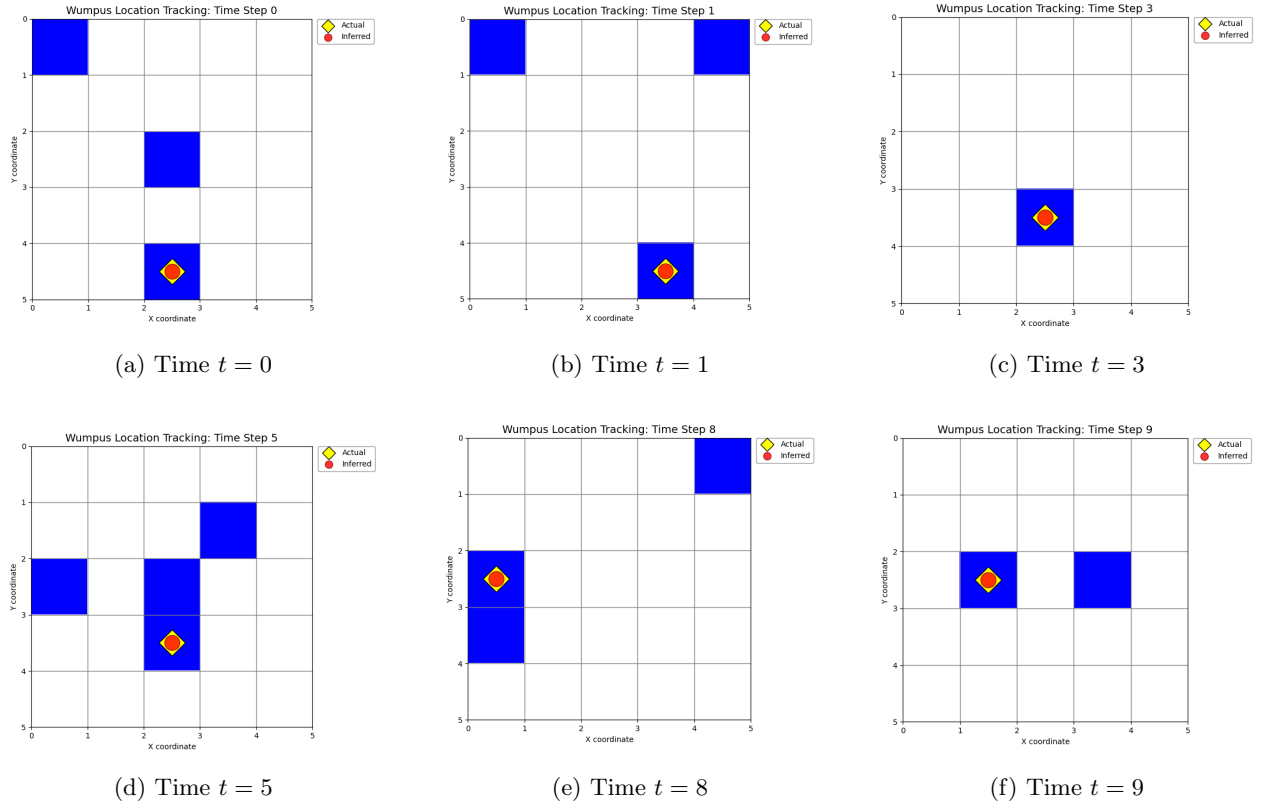


Figure 3: Dataset 1 tracking performance across select time steps. The grid displays binary detection data, where the **yellow star** represents the ground-truth wumpus location and the **red dot** represents the location inferred by the MAP algorithm.

References

- [1] Daniel Jurafsky and James H. Martin. *Hidden Markov Models*. 2023. Appendix A.